

Task 1- A Quick Guide:

1. Domain Description

Emergency Response Coordination System (ERCS) is a system that coordinates emergency units (fire engines, ambulances, police, rescue teams) during incidents like wildfires, floods and building collapses.

The Problem It Solves

Problem	How ERCS Solves It
Units are organized hierarchically	Composite - Incident Command → Sections → Teams → Individual Units
Units change behavior during missions	State - Units transition: STANDBY → EN ROUTE → ON SCENE → STABILIZING → DONE
Add extra capabilities to units at runtime	Decorator - Wrap units with K9, Aerial, or Technical Rescue capabilities
Need to traverse the hierarchy in different ways	Iterator - Find all units, available units, or filter by type

2. The GoF Participant Mapping:

Composite Pattern

GoF Role Our Class

Component EmergencyComponent (abstract base)

Leaf EmergencyUnit, FireEngine, Ambulance, PoliceUnit, RescueUnit

Composite EmergencyGroup

State Pattern

GoF Role	Our Class
----------	-----------

Context	EmergencyUnit (holds current state)
---------	-------------------------------------

State	UnitState (abstract)
-------	----------------------

ConcreteState	StandbyState, EnRouteState, OnSceneState, StabilizingState, TransportingState, DoneSta
---------------	--

Decorator Pattern

GoF Role	Our Class
----------	-----------

Component	EmergencyComponent
-----------	--------------------

ConcreteComponent	Any EmergencyUnit or EmergencyGroup instance
-------------------	--

Decorator	UnitDecorator (abstract)
-----------	--------------------------

ConcreteDecorator	K9Decorator, AerialDecorator, TechnicalRescueDecorator
-------------------	--

Iterator Pattern

GoF Role	Our Class
----------	-----------

Iterator	EmergencyIterator (abstract)
----------	------------------------------

ConcreteIterator	AllUnitsIterator, AvailableUnitsIterator, TypeFilterIterator
------------------	--

Aggregate	EmergencyComponent (declares factory methods)
-----------	---

ConcreteAggregate	EmergencyGroup, EmergencyUnit
-------------------	-------------------------------

3. Important Design & Ownership Decisions:

Decision 1: Single EmergencyGroup (Not 3 Subclasses)

One EmergencyGroup class that can contain any EmergencyComponent (another group OR a unit).

Why: True recursion. A group can contain another group, which can contain another group... This is more faithful to the Composite pattern than hard-coding 3 levels.

Example:

EmergencyGroup("Incident Command") ← Level 1

└─ EmergencyGroup("Operations") ← Level 2

└─ EmergencyGroup("StrikeTeam") ← Level 3

└─ FireEngine("Engine 1") ← Level 4

Decision 2: States as Shared Singletons

Each state is a singleton (getInstance()). Units just point to the shared instance.

Why: States have no instance data, so they can be shared. This prevents memory leaks and double-deletion risks.

// How units use states:

unit->setState(StandbyState::getInstance());

unit->setState(EnRouteState::getInstance());

Decision 3: Executor/Advance/Cancel as Shared Operations

Three core operations that work across ALL patterns:

Operation	On Composite	On Leaf	On Decorator
execute()	Loops over children	Delegates to State	Delegates + adds own behavior
advance()	Loops over children	Tells State to transition	Delegates
cancel()	Loops over children	Tells State to cancel	Delegates

Now all four patterns work together through these three operations.

Decision 5: Ownership Rules

Who	Owns What	Deletes What
EmergencyGroup	children vector	All children in destructor
UnitDecorator	wrapped pointer	wrapped in destructor
EmergencyUnit	currentState pointer	NOTHING - states are singletons, don't delete
Client (main.cpp)	EmergencyIterator*	Must delete after use